Two Arms, One Controller

Why the G1-D Hand Moves Faster than the UR5e Hand

Tri Wahyu Guntara
Controller notes on the UWLab OmniReset code base

1 What the Policy Commands

Every tenth of a second the OmniReset policy [1] writes six numbers. The first three ask the hand to move along the x, y and z axes of the robot base, and the last three ask it to turn about those same axes. The policy never names a joint angle or a torque. It names a small change of the hand pose, and a controller underneath turns that wish into motor torques.

The six numbers first pass through a scale. Both robots use the same one, 20 millimetres per unit for each translation and 0.2 radians per unit for the turn about z. The controller then builds a target pose by adding the scaled change to the pose the hand has *right now*,

$$\boldsymbol{x}_{\text{target}} = \boldsymbol{x}_{\text{measured}} + \boldsymbol{s} \odot \boldsymbol{a}, \tag{1}$$

where $\boldsymbol{a}$ is the policy output and $\boldsymbol{s}$ is the scale. Two details matter for the rest of this note. The target is rebuilt from the measured pose at every policy step, so the controller only ever chases a point one action ahead of the hand. And nothing limits the size of $\boldsymbol{a}$. A policy that outputs 5 asks for 100 millimetres just as easily as one that outputs 1 asks for 20.

Between two policy steps the physics engine advances twelve times, at 120 hertz. At each of those twelve substeps the controller measures the arm again and recomputes the torques, while the target stays fixed. Figure 1 shows the loop.

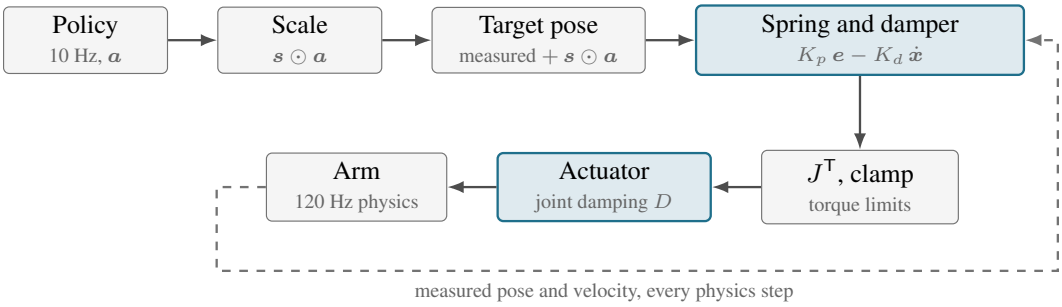


Figure 1. The controller of both robots. The policy runs at 10 hertz and everything inside the loop runs at 120 hertz. The two outlined blocks hold the settings in which the robots differ.

2 A Spring, a Damper and a Jacobian

The controller behaves like an invisible spring stretched between the hand and the target, with a shock absorber mounted beside it [2]. The spring pulls the hand toward the target with a force that grows with the distance. The shock absorber, which engineers call a damper, pushes against the hand with a force that grows with its speed. For the force $\boldsymbol{F}$ on the hand the law reads

$$\boldsymbol{F} = K_p\, \boldsymbol{e} - K_d\, \dot{\boldsymbol{x}}, \tag{2}$$

where $\boldsymbol{e}$ is the pose error, the gap between target and hand, and $\dot{\boldsymbol{x}}$ is the hand velocity. The stiffness K_p sets how hard the spring pulls, and the damping K_d sets how hard the shock absorber resists motion.

A force at the hand is not something a motor can produce directly. The Jacobian J, a matrix that relates joint speeds to hand speed, translates the force into one torque per joint. The controller also adds a small posture term for the seven-jointed G1-D, which keeps the elbow near a preferred angle without moving the hand, and the actuator adds its own joint damping D. The full torque on the joints is therefore

$$\boldsymbol{\tau} = J^{\mathsf{T}}\big(K_p\,\boldsymbol{e} - K_d\,J\dot{\boldsymbol{q}}\big) + \boldsymbol{\tau}_{\text{posture}} - D\,\dot{\boldsymbol{q}}, \tag{3}$$

where $\dot{\boldsymbol{q}}$ collects the joint speeds. The torques are then clipped to each motor's limit. Notice what the law leaves out. It never accounts for how heavy the arm is. A heavy arm and a light arm receive the same force for the same error, and they respond very differently to it. That omission is the root of this whole story.

The configuration files do not store K_d directly. They store a number called the damping ratio, written ζ, and convert it with the rule

$$K_d = 2\,\zeta\sqrt{K_p}. \tag{4}$$

This rule is exact only for a hand that weighs one kilogram. Section 4 shows what it means for real arms, and Figure 2 pictures the two hands as masses on springs.

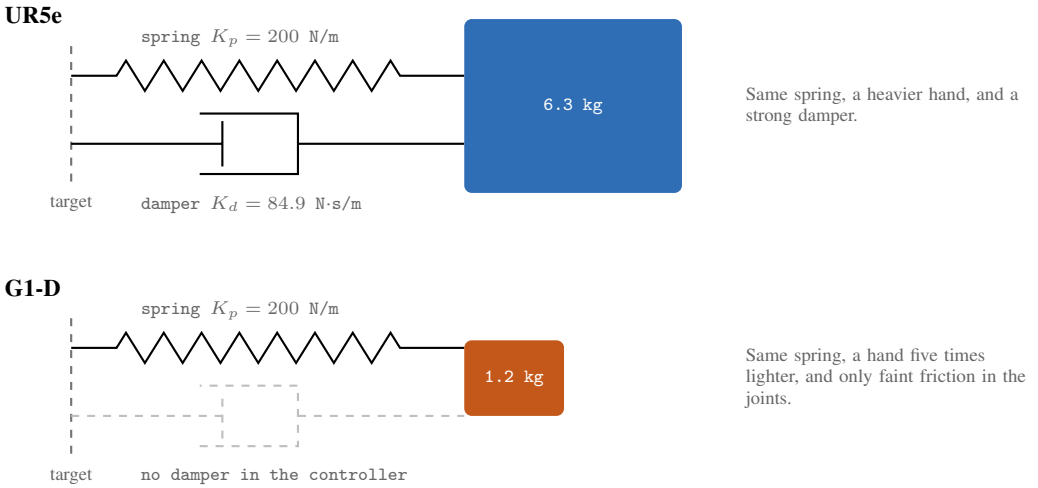


Figure 2. The two hands seen as masses on springs, moving sideways along y in the test pose of Section 5. The masses are the effective task-space inertia measured from the simulator.

3 Where the Two Robots Differ

The G1-D configuration was written to mirror the UR5e recipe, and most of its numbers are indeed identical. Table 1 lists every setting that reaches the controller or the actuator, with the rows that differ shaded.

Setting	UR5e	G1-D with Dex1
Arm joints under control	6	7
Action scale per unit	20 mm, 0.2 rad about z	20 mm, 0.2 rad about z
Stiffness K_p, translation and rotation	200 N/m, 3 N·m/rad	200 N/m, 3 N·m/rad
Damping ratio ζ, translation and rotation	3 and 1	0 and 0
Controller damping K_d	84.9 N·s/m, 3.46 N·m·s/rad	none
Actuator joint damping D	none	1 N·m·s/rad on every joint
Posture task in the null space	none	stiffness 2, damping 2.83
Torque limits	150 N·m shoulder and elbow, 28 N·m wrist	25 N·m, 5 N·m wrist pitch and yaw
Joint speed limits	1.57 rad/s large joints, 3.14 rad/s wrist	37 rad/s, 22 rad/s wrist
Effective hand mass along y at the test pose	6.3 kg	1.2 kg
Inertia of the last wrist joint	0.0167 kg·m^2	0.0041 kg·m^2
Physics rate and policy rate	120 Hz and 10 Hz	120 Hz and 10 Hz
Action clipping	none	none

Table 1. Controller and actuator settings of the two robots in the stage 1 training configuration.

Three differences dominate. The G1-D controller has no damper at all, the G1-D hand is about five times lighter, and its joint speed limits are so high that they never come into play. The UR5e speed limit of 1.57 radians per second, on the other hand, quietly caps how fast its large joints can swing.

The settings come straight from three places in the code. The UR5e action is defined as follows.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/actions.py lines 15 to 23

```
UR5E_ROBOTIQ_2F85_RELATIVE_OSC = RelCartesianOSCActionCfg(
    asset_name="robot",
    joint_names=["shoulder.*", "elbow.*", "wrist.*"],
    body_name="wrist_3_link",
    scale_xyz_axisangle=(0.02, 0.02, 0.02, 0.02, 0.02, 0.2),
    motion_stiffness=(200.0, 200.0, 200.0, 3.0, 3.0, 3.0),
    motion_damping_ratio=(3.0, 3.0, 3.0, 1.0, 1.0, 1.0),
    torque_limit=(150.0, 150.0, 150.0, 28.0, 28.0, 28.0),
)
```

The G1-D action copies the stiffness and the scale but sets every damping ratio to zero, and its comment records why.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/g1d_dex1/actions.py lines 27 to 42

```
G1D_DEX1_RELATIVE_OSC = RelCartesianOSCActionCfg(
    asset_name="robot",
    joint_names=G1D_ARM_JOINT_NAMES,
    body_name=G1D_EE_BODY_NAME,
    scale_xyz_axisangle=(0.02, 0.02, 0.02, 0.02, 0.02, 0.2),
    motion_stiffness=(200.0, 200.0, 200.0, 3.0, 3.0, 3.0),
    # No explicit task-space damping: it is unstable on the light G1-D wrist links; the damping is
      provided
    # implicitly by the arm actuator (see IMPLICIT_G1D_DEX1 in uwlab_assets).
    motion_damping_ratio=(0.0, 0.0, 0.0, 0.0, 0.0, 0.0),
    torque_limit=G1D_TORQUE_LIMIT,
    jacobian_fn=G1D_JACOBIAN_FN,
    # 7-DoF arm: hold the default elbow configuration in the null space of the task
    nullspace_stiffness=2.0,
    nullspace_damping_ratio=1.0,
    nullspace_joint_pos=G1D_DEFAULT_ARM_JOINT_POS,
)
```

The damping that the G1-D does have lives in its arm actuator, one newton metre per radian per second on each joint.

source/uwlab_assets/uwlab_assets/robots/unitree_g1d_dex1/g1d_dex1.py lines 136 to 150

```
IMPLICIT_G1D_DEX1.actuators = {
    "arm": ImplicitActuatorCfg(
        joint_names_expr=G1D_ARM_JOINT_REGEX,
        stiffness=0.0,
        # Joint damping lives here (PhysX applies it implicitly, stable at any gain) instead of in the USC: the
        # wrist links weigh ~85 g (I ~ 5e-4 kg m^2 about the roll axis), and an explicit task-space damping torque
        # of a few N m s/rad at 120 Hz overshoots the velocity each substep and diverges within a handful of
        # substeps (verified with scripts_v2/tools/debug_reset_env-style probes on 2026-10-26). The UR5e gets away
        # with explicit damping only because its wrist links are ~10x heavier.
        damping=1.0,
        effort_limit_sim=G1D_EFFORT_LIMITS,
        velocity_limit_sim=G1D_VELOCITY_LIMITS,
    ),
    "gripper": DEX1_GRIPPER_ACTUATOR,
}
```

The inertia figures in Table 1 come from the simulator itself. The effective hand mass is the task-space inertia, a quantity that Section 4 introduces, evaluated at the default arm pose of each robot.

4 One Mass on a Spring

A whole arm is complicated, but along any single direction it behaves much like one mass. That mass is called the task-space inertia and is written Λ. It answers a simple question. If you push the hand with one newton, how quickly does it start to move? Physics gives the answer through the joint inertia matrix M and the Jacobian,

$$\Lambda = \left(J M^{-1} J^{\mathsf{T}}\right)^{-1}. \tag{5}$$

With that single mass in hand, the controller reduces to the textbook mass on a spring with a damper,

$$\Lambda\, \ddot{x} = K_p\, (x_{\text{target}} - x) - K_d\, \dot{x}. \tag{6}$$

Two numbers describe how such a system moves. The natural frequency tells how quickly it reacts, and the true damping ratio tells whether it overshoots,

$$\omega_n = \sqrt{\frac{K_p}{\Lambda}}, \qquad \zeta_{\text{true}} = \frac{K_d}{2\sqrt{K_p\, \Lambda}}. \tag{7}$$

For the UR5e moving along y, with a mass of 6.3 kilograms, the natural frequency is 5.6 radians per second and the true damping ratio is 1.2. The setting of 3 in the configuration file therefore produces a hand that is a little more than critically damped, which is exactly the calm, no-overshoot motion the UR5e shows. For the G1-D, with a mass of 1.2 kilograms, the same spring gives a natural frequency of 13 radians per second, more than twice as fast, and the controller adds no damping at all.

The speed the hand settles at. Because the target is rebuilt from the measured pose at every policy step, a policy that holds the same action keeps the spring stretched by the same amount, step after step. The hand then stops accelerating once the damper pushes back as hard as the spring pulls. Setting the two forces equal gives the cruising speed,

$$v_{\text{cruise}} = \frac{K_p\, s\, a}{K_d}. \tag{8}$$

For the UR5e with an action of one, this is 200 times 0.02 divided by 84.9, or 47 millimetres per second. Notice that the hand mass has dropped out. Once the damper is present, the cruising speed depends only on the gains, the scale and the action. Without a damper the formula breaks down, and the only thing left to stop the G1-D hand is the faint joint damping in its actuator. The cruising speed also grows in direct proportion to the action a. Keep this in mind for Section 7, because the policy is free to choose a.

5 Measuring the Two Arms

Theory is only half of the argument, so both arms were tested in simulation with the training configuration. Each arm started from its default pose with gravity off, as in training. The test then gave one action of plus one for a single policy step, waited, held the same action for one full second, and finally released it. The hand was recorded at every physics step. Figure 3 shows the sideways motion.

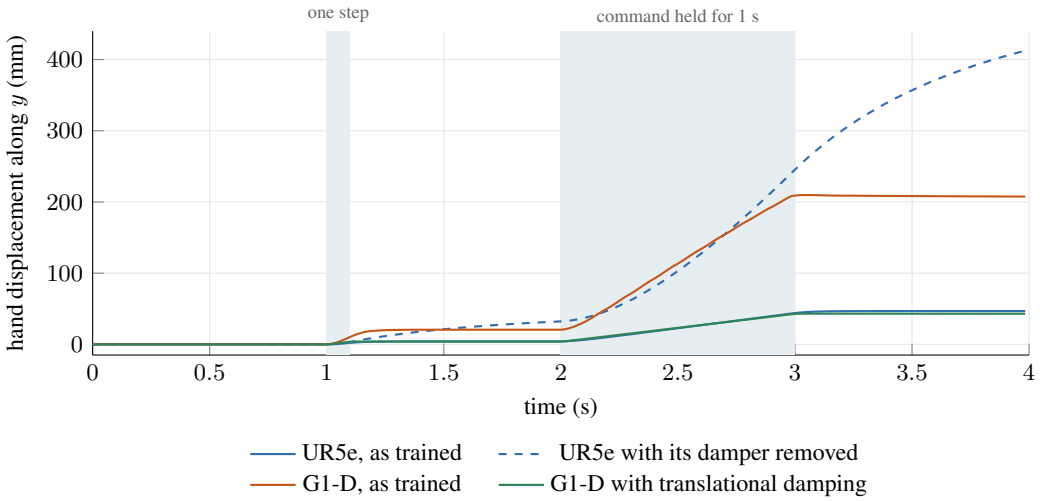


Figure 3. Sideways motion of the hand along y. The shaded bands mark when the command was active. With its damper, the UR5e hand cruises at 43 millimetres per second, close to the 47 that Equation (8) predicts. The G1-D hand reaches 224 millimetres per second, five times faster. Remove the damper from the UR5e and it runs even faster, then keeps sliding for 17 centimetres after the command ends. Give the G1-D the UR5e translational damping and it falls in line with the UR5e.

The figure carries the central result. The difference in speed comes almost entirely from the missing damper. The UR5e without its damper runs even faster than the G1-D, because nothing but its joint speed limits stand in its way. The G1-D at least has its faint joint damping, which is also why it stops quickly after the command ends rather than coasting. Table 2 collects all six directions.

Direction	UR5e as trained	G1-D as trained	G1-D, translational damping	UR5e, no damper
x, peak speed	43 mm/s	24 mm/s	13 mm/s	226 mm/s
y, peak speed	43 mm/s	224 mm/s	42 mm/s	321 mm/s
z, peak speed	43 mm/s	240 mm/s	42 mm/s	330 mm/s
Turn about x, peak rate	1.0 °/s	8.4 °/s	8.1 °/s	24 °/s
Turn about y, peak rate	1.0 °/s	5.5 °/s	3.2 °/s	19 °/s
Turn about z, peak rate	17 °/s	34 °/s	22 °/s	unstable

Table 2. Peak hand speed while an action of one is held for one second, measured at the default arm pose.

The G1-D x direction is the exception. In the default pose, moving the hand forward along x appears to need large joint speeds, so even the faint joint damping slows it a great deal. The large effective mass in that direction, about 8 kilograms, points the same way. Real episodes visit many poses, so this direction is not always slow. Every number here belongs to one pose and would change somewhat in another.

Rotation tells a similar story. The G1-D turns two to eight times faster than the UR5e, and the translational damper alone barely changes that. Figure 4 shows the turn about z and introduces the problem that the next section explains.

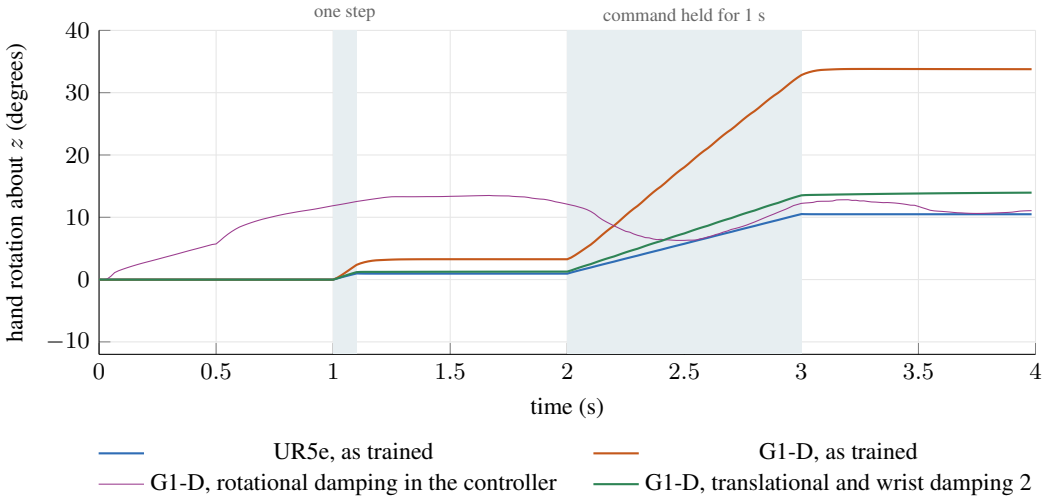


Figure 4. Rotation of the hand about z. Adding rotational damping inside the controller, as the UR5e does, makes the G1-D wrist chatter violently, and the commanded turn is lost in the shaking. The chatter begins at rest, before any command arrives, which shows that the damping itself drives it. Adding the damping to the wrist joints in the actuator instead slows the turn to 14 degrees per second, close to the UR5e, with no chatter.

6 Why the G1-D Lost Its Damper

The missing damper was a deliberate choice. The comment in the G1-D action configuration records that explicit damping in the controller made the light wrist unstable. Figure 4 confirms that warning, and a short calculation explains it.

The controller computes its damping torque from the velocity it measured at the start of a physics step, then holds that torque for the whole step of $\Delta t = 1/120$ of a second. Consider a single joint with inertia I spinning at speed v. After one step the damping torque has changed the speed to

$$v_{k+1} = \left(1 - \frac{K_d\,\Delta t}{I}\right) v_k. \tag{9}$$

If the bracket lies between minus one and one, the speed shrinks every step and the joint settles. If the damping is too strong for the inertia, the bracket drops below minus one, and the joint reverses direction faster than it was moving before. Each step then makes the motion larger. The stability condition is therefore

$$\frac{K_d\,\Delta t}{I} < 2. \tag{10}$$

The rotational damping of the UR5e recipe, 3.46 newton metres per radian per second, acts almost directly on the last wrist joint. Figure 5 compares the ratio for each wrist joint of the two robots.

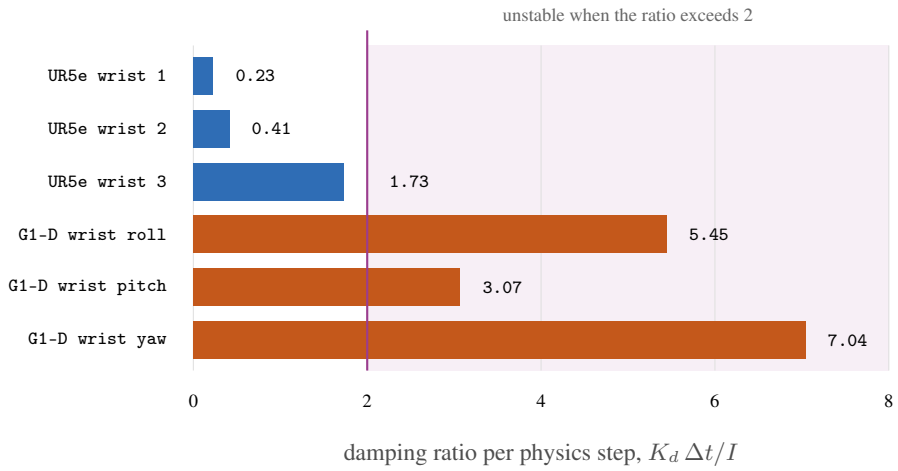


Figure 5. The stability ratio of the UR5e rotational damping applied to each wrist joint. The UR5e stays below 2, with its last joint at 1.73, close to the edge. Every G1-D wrist joint lies above 2, and the wrist yaw sits at 7. The inertias are the diagonal entries of the joint inertia matrix at the default pose, so the bars give a fair estimate rather than an exact bound.

Translational damping is a different matter. A force at the hand reaches the wrist joints through short lever arms, so the damping that the wrist feels from it is small. The measurements agree. The G1-D with the full UR5e translational damping ran smoothly in every direction, with no sign of chatter.

The actuator joint damping D also escapes the problem. The physics engine solves it as part of the motion rather than applying it as a fixed torque, which keeps it stable at any value. That is why the original author moved the G1-D damping there, and it is also why that location suits the wrist rotation.

7 The Policy Pushes Back

So far the action was held at one. A trained policy is under no such restraint. Neither robot clips its actions, and both policies have learned to command far more than one. Table 3 shows the size of the actions that three trained policies produced during 160 steps of peg insertion in 32 environments.

Policy	Translation, 90th pct.	Rotation, 90th pct.	Share above one	Hand speed, 90th and 99th pct.
UR5e peg, full scale	2.8 to 4.3	6.4 to 19	23 to 69 %	0.29 and 0.56 m/s
G1-D peg, full scale	1.5 to 4.3	2.7 to 9.2	18 to 90 %	0.30 and 0.94 m/s
G1-D peg, quarter scale	5.0 to 9.8	11 to 13	34 to 68 %	0.26 and 0.55 m/s

Table 3. Size of the arm actions of three trained policies, per axis, and the resulting hand speed. The action columns give the range over the three axes of each kind.

The last row is the important one. When the action scale was cut to a quarter, the policy simply learned to output larger actions, about two to four times larger than before. The quarter-scale policy now moves its hand at nearly the same speeds as the UR5e policy. Its fastest moments dropped from 0.94 to 0.55 metres per second, but its typical fast motion barely changed. Figure 6 shows the same robot with an action of exactly one.

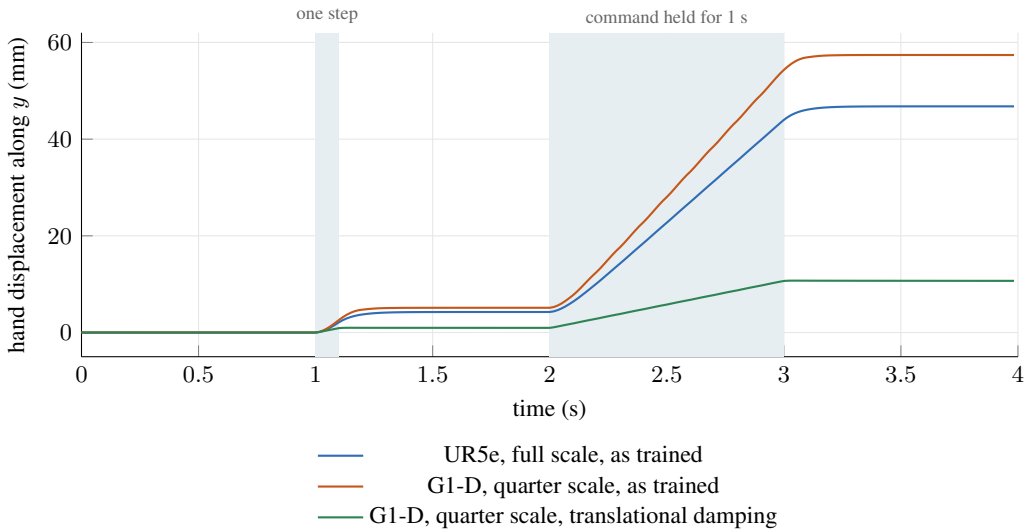


Figure 6. The quarter-scale G1-D with an action of exactly one. Measured this way it looks calm, at 56 millimetres per second, and translational damping slows it further to 10. The trained policy does not command one, though. It commands five to ten, and the speed grows in proportion.

This explains why the quarter-scale robot still looked too fast. A policy optimizes for reward, and reaching the peg quickly earns reward. If the controller makes each unit of action smaller, the policy asks for more units. Equation (8) makes the trade explicit. Any change to the stiffness, the scale or the damping is undone by a matching change in a, as long as a is free.

The UR5e policy is no gentler by nature. Its hand reaches 0.56 metres per second in its fastest moments, close to what its 1.57 radian per second joint limit allows. It looks calm partly because its damper keeps every motion smooth and partly because it is a large robot, so the same speed looks slower on it.

8 A Recipe for a Calmer G1-D

Damping is the right instinct, and it belongs in the recipe. It makes the G1-D respond like the UR5e for each unit of action, and it removes the snap at the start and end of each motion. On its own, however, it will not slow the trained robot down, because the policy will learn to push harder. A calmer G1-D needs three changes together, followed by retraining.

1. **Add translational damping in the controller.** Set the translational damping ratio to 3, as on the UR5e, and leave the rotational ratio at 0. The measurements show this is stable on the G1-D and that it brings the cruising speed down to the UR5e value of about 43 millimetres per second for each unit of action.
2. **Damp the wrist rotation in the actuator.** Raise the joint damping of the three wrist joints from 1 to about 2 newton metres per radian per second, and keep it at 1 for the shoulder and elbow. The physics engine handles this damping implicitly, so it stays stable. In the test, the turn about z slowed from 34 to 14 degrees per second, close to the UR5e.
3. **Bound the action.** Clip the policy output so that the speed formula has a ceiling the policy cannot negotiate away. With a clip at c, the fastest the hand can cruise becomes

$$v_{\max} = \frac{K_p \, s \, c}{K_d} \approx 0.047 \, c \text{ m/s} \tag{11}$$

at the full scale with the damping of the first step. A clip of 1 gives 47 millimetres per second, which would make a 30 centimetre reach take more than six seconds of a sixteen

second episode. A clip of 4 gives about 0.19 metres per second. The clip should be chosen from the speed you want, and the same reasoning sets the rotational ceiling. Section 9 derives a value from a speed that was actually watched.

Table 4 collects the three changes, the setting each one touches and the file that holds it.

Change	Setting and file	Now	Proposed
Translational damping	`motion_damping_ratio` of `G1D_DEX1_RELATIVE_OSC` in `config/g1d_dex1/actions.py`	0 on all six axes	3 on x, y and z, 0 on the rotations
Wrist damping	`damping` of the arm actuator in `IMPLICIT_G1D_DEX1`, file `unitree_g1d_dex1/g1d_dex1.py`	1 on all seven joints	2 on the three wrist joints, 1 elsewhere, in a second actuator group
Action bound	`clip_actions` of `G1dDex1_PPORunnerCfg` in `config/g1d_dex1/agents/rsl_rl_cfg.py`	not set	chosen from the target speed, about 7 (Section 9)

Table 4. The proposed changes. None of them is applied in the repository yet.

Two cautions apply. First, the action clip reshapes what the policy can learn, so it calls for a fresh training run rather than a resumed one, and the episode length should be checked against the slower ceiling. Second, the reset-state datasets were recorded with the old controller. They describe poses rather than motions, so they remain valid, but the grasp hold test during recording uses the controller too and may give slightly different yields.

A cheaper check is possible before committing to a full run. Train for a few hundred iterations with each candidate clip, then measure the hand speed percentiles as in Section 7. The 99th percentile is the number that best matches the impression of a robot that moves too fast.

9 Old Policies on the New Controller

Before retraining anything, the two full-scale G1-D policies were run with the controller of Section 8. The peg policy at iteration 2200 and the cube policy at iteration 1700 were both trained with the old controller. They received the translational damping ratio of 3 and the wrist joint damping of 2, but no action clip, and nothing in the networks changed. The episodes started from the same reset states as the earlier rollouts, so each episode can be compared with its counterpart. Table 5 gives the outcome.

Policy	Old controller	New controller
Peg insertion, iteration 2200	9 of 10	7 of 10
Cube stacking, iteration 1700	10 of 10	5 of 10

Table 5. Successful episodes out of ten for policies trained with the old controller.

The drop is expected. Each policy learned how far the hand travels for a given action, and the new controller moves the hand about five times less for the same action along y and z. A policy that expects the hand to arrive in two steps now finds it a fifth of the way there. The peg policy copes better than the cube policy, perhaps because insertion already rewards slow, careful motion near the hole.

The motion looks much calmer, and the numbers agree. Table 6 compares the hand speed of the same policies, measured over 160 steps in 32 environments, with and without the new controller.

Policy	Controller	Hand speed, 50th, 90th, 99th pct.	Turn rate, 90th, 99th pct.	Translation action, 90th pct.
Peg insertion	old	0.006, 0.30, 0.94 m/s	1.18, 3.38 rad/s	1.5 to 4.3
Peg insertion	new	0.039, 0.18, 0.32 m/s	0.78, 1.50 rad/s	3.0 to 4.6
Cube stacking	old	0.004, 0.24, 0.86 m/s	0.87, 3.22 rad/s	0.8 to 0.9
Cube stacking	new	0.008, 0.17, 0.32 m/s	0.72, 1.36 rad/s	2.8 to 3.1

Table 6. Hand speed, turn rate and action size of the same two policies with the old and the new controller.

The fastest moments, the 99th percentile, fell by almost two thirds, from about 0.9 to 0.32 metres per second, and the fastest turns fell by more than half. Those fast moments are what the eye notices, so this is the change that made the robot look calmer.

The last column shows something important. Even without retraining, the policies already push harder. The cube policy roughly tripled its typical action, and the share of its translation actions above one rose from about 9 to about 24 percent. Nothing in the network changed. The policy reacts to what it sees, and a hand that lags behind leaves a larger error in view, so the policy answers with a larger action. This is the closed-loop version of the effect in Section 7, and retraining would only strengthen it.

Keeping this speed after retraining. A policy retrained on the new controller will learn to use larger actions on purpose, and the speed would creep back up. The action clip of Section 8 prevents that. To keep the speed that was watched here, the cruising ceiling should sit near the fastest moments observed, about 0.32 metres per second. With the damping of Section 8 at full scale, each unit of action buys about 0.047 metres per second of cruising speed, so

$$c \approx \frac{0.32}{0.047} \approx 7. \tag{12}$$

The rotation agrees. With wrist damping of 2, one unit of action turns the hand at about 14 degrees, or 0.24 radians, per second about z, and a clip of 7 allows about 1.7 radians per second, close to the 1.4 to 1.5 observed. A clip of 7 therefore preserves the motion that was watched while removing the room to speed up again.

Three limits apply to this estimate. The cruising speeds come from one arm pose, and the forward x direction was much slower there. A clip bounds the steady speed, not the brief surge at the start of a motion, so short peaks a little above the ceiling remain possible. And the success rates above belong to policies that were never trained on this controller, so they say nothing about what a retrained policy will reach. The rollouts are in `logs/episode_renders/g1d_peg_newctl_s0`, `g1d_peg_newctl_s1` and `g1d_cube_newctl`.

How the Numbers Were Measured

The measurements were taken on 2 November 2026 with the working tree of the `feature/forge-omnireset` branch. The step tests come from `logs/probe/osc_step.py`, run on the play tasks of both robots with one damping variant and one axis per environment, all starting from the default arm pose with gravity off, as in training. The action statistics come from `logs/probe/action_stats.py`, run on UR5e peg iteration 2999, G1-D peg iteration 2200 and G1-D quarter-scale peg iteration 1300, with deterministic actions, and again with the controller of Section 8 for Section 9. The task-space inertia was computed from the joint inertia matrix of the simulator and the analytical Jacobian of each arm.

References

[1] UW Lab. Emergent dexterity via diverse resets and large-scale reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.15789.

[2] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.